

```
In [11]: import os
import time
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

print("Environment ready. Libraries imported successfully.")
```

Environment ready. Libraries imported successfully.

```
In [12]: # Load dataset
import pandas as pd
df = pd.read_csv("combined_data.csv.xls")

# Replace missing values with the mean of each column
df = df.fillna(df.mean())

# Align features
FEATURES = ["Ax", "Ay", "Az", "Gx", "Gy", "Gz", "Temp1", "Temp2", "Flag", "Capteur1_cm", "Capteur2_cm"]

print("Dataset loaded and cleaned.")
print(df.head())
```

Dataset loaded and cleaned.

	Timestamp	Ax	Ay	Az	Gx	Gy	Gz	Col8	Temp1	Temp2	Flag	\
0	113251	-0.58	0.20	-10.58	0.24	0.14	0.05	1	38.0	31.0	0	
1	113326	-0.57	0.28	-10.56	0.10	-0.06	0.07	1	38.0	31.0	0	
2	113401	-0.60	0.31	-10.52	0.01	-0.01	0.05	1	38.0	31.0	0	
3	113477	-0.57	0.34	-10.54	0.03	0.03	0.11	1	38.0	31.0	0	
4	113552	-0.70	0.23	-10.54	-0.10	0.42	-0.10	1	38.0	31.0	0	

	Capteur1_cm	Capteur2_cm
0	12.25	4.39
1	12.71	4.30
2	12.73	4.39
3	12.67	4.39
4	12.79	4.37

```
In [13]: def assign_label(row):
    if row["Temp1"] > 50 or row["Temp2"] > 50:
        return 2 # DANGER
    elif row["Capteur1_cm"] < 2 or row["Capteur2_cm"] < 2:
        return 3 # EVACUATE
    else:
        return 0 # SAFE

df["risk_label"] = df.apply(assign_label, axis=1)

print("Labels generated successfully!")
print(df[["Temp1", "Temp2", "Capteur1_cm", "Capteur2_cm", "risk_label"]].head())
```

Labels generated successfully!

	Temp1	Temp2	Capteur1_cm	Capteur2_cm	risk_label
0	38.0	31.0	12.25	4.39	0
1	38.0	31.0	12.71	4.30	0
2	38.0	31.0	12.73	4.39	0
3	38.0	31.0	12.67	4.39	0
4	38.0	31.0	12.79	4.37	0

```
In [14]: X = df[FEATURES]
y = df["risk_label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Data split complete.")
```

Data split complete.

```
In [18]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler = StandardScaler()

# Fit on training data and transform it
X_train_scaled = scaler.fit_transform(X_train)

# Transform the test data using the same scaler
X_test_scaled = scaler.transform(X_test)
```

```
In [17]: model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

```
print("Model training complete!")
```

Model training complete!

```
In [19]: joblib.dump(model, "evacuation_rf_model.pkl")
joblib.dump(scaler, "evacuation_scaler.pkl")

print("Model and scaler saved successfully!")
```

Model and scaler saved successfully!

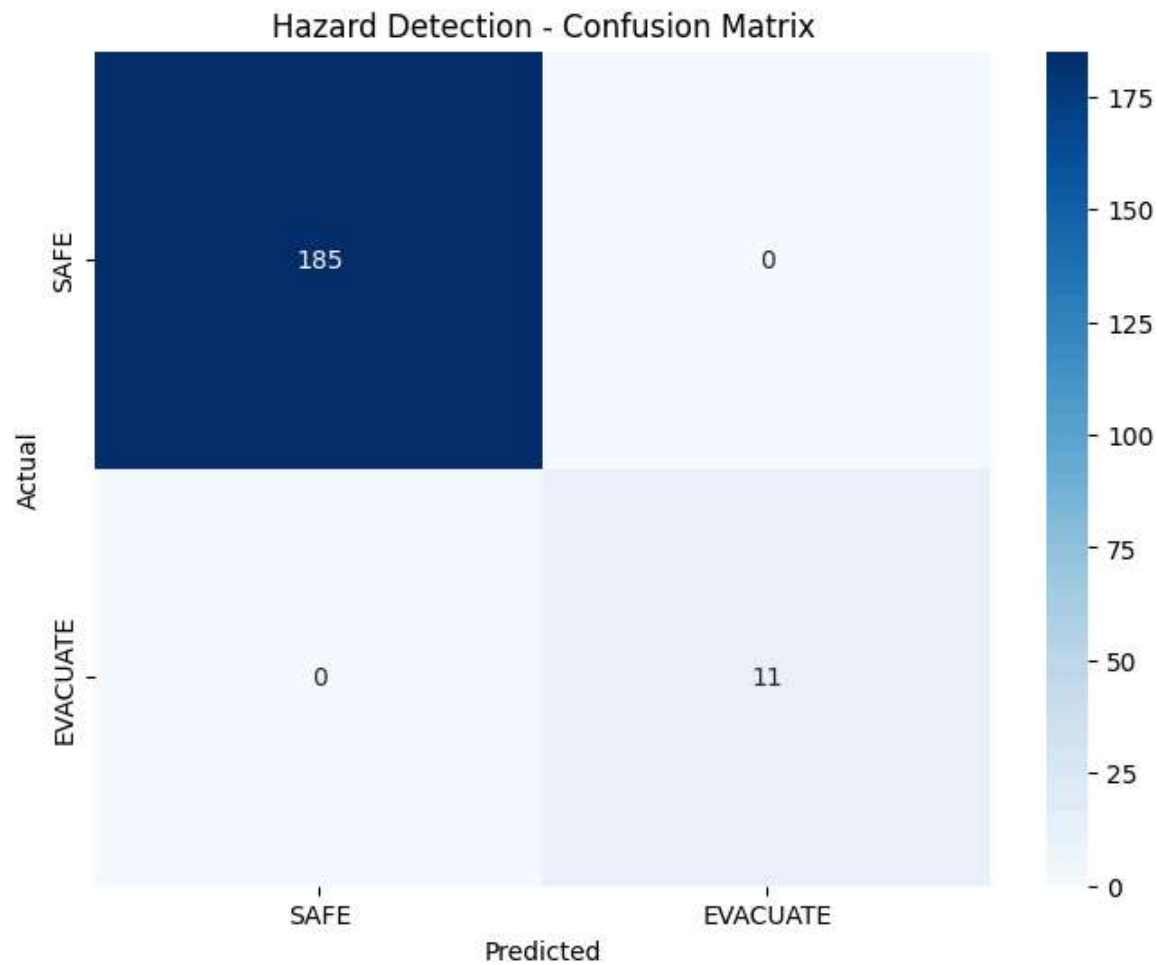
```
In [20]: labels = sorted(y.unique()) # actual labels present in y
label_map = {0:"SAFE", 1:"WATCH", 2:"DANGER", 3:"EVACUATE"}
target_names = [label_map[l] for l in labels]

print("=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred, target_names=target_names))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=target_names, yticklabels=target_names)
plt.title("Hazard Detection - Confusion Matrix")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()
```

```
=== CLASSIFICATION REPORT ===
```

	precision	recall	f1-score	support
SAFE	1.00	1.00	1.00	185
EVACUATE	1.00	1.00	1.00	11
accuracy			1.00	196
macro avg	1.00	1.00	1.00	196
weighted avg	1.00	1.00	1.00	196



```
In [21]: deployed_model = joblib.load("evacuation_rf_model.pkl")
         deployed_scaler = joblib.load("evacuation_scaler.pkl")

         status_labels = {0: "SAFE", 1: "WATCH", 2: "DANGER", 3: "EVACUATE"}

         def live_monitor_simulation(seconds=10, path_threshold=1.0):
             print("--- REAL-TIME STRUCTURAL MONITORING ---")
             for i in range(seconds):
                 # Example simulated input
                 Ax, Ay, Az = np.random.uniform(-1,1), np.random.uniform(-1,1), np.random.uniform(-12,-9)
                 Gx, Gy, Gz = np.random.uniform(-5,5), np.random.uniform(-5,5), np.random.uniform(-5,5)
                 Temp1, Temp2 = np.random.uniform(20,40), np.random.uniform(20,40)
```

```

Flag = np.random.choice([0,1])
Capteur1_cm = np.random.uniform(0.0, 12.0)
Capteur2_cm = np.random.uniform(0.0, 12.0)

current_reading = pd.DataFrame([[Ax,Ay,Az,Gx,Gy,Gz,Temp1,Temp2,Flag,Capteur1_cm,Capteur2_cm]],
                                columns=FEATURES)
current_scaled = deployed_scaler.transform(current_reading)
prediction = deployed_model.predict(current_scaled)[0]
status = status_labels[prediction]

print(f"Time {i}s | Ax:{Ax:.2f} Ay:{Ay:.2f} Az:{Az:.2f} | Temp:{Temp1:.1f}/{Temp2:.1f}°C | STATUS: {status}")

if status == "EVACUATE":
    if Capteur1_cm <= path_threshold:
        print("⚠️ Path A blocked → Choosing Path B")
    else:
        print("✅ Path A clear → Choosing Path A")
    print("🚨 EMERGENCY WARNING: ACTIVATING SMART BUILDING ALARMS 🚨")
    time.sleep(0.5)

# Run simulation
live_monitor_simulation()

```

```

--- REAL-TIME STRUCTURAL MONITORING ---
Time 0s | Ax:0.86 Ay:-0.28 Az:-11.26 | Temp:35.2/31.0°C | STATUS: SAFE
Time 1s | Ax:0.79 Ay:0.03 Az:-9.11 | Temp:34.5/21.3°C | STATUS: SAFE
Time 2s | Ax:0.70 Ay:-0.41 Az:-10.38 | Temp:25.2/24.5°C | STATUS: SAFE
Time 3s | Ax:0.67 Ay:-0.52 Az:-10.93 | Temp:23.3/20.6°C | STATUS: SAFE
Time 4s | Ax:-0.12 Ay:-0.03 Az:-11.78 | Temp:24.5/33.5°C | STATUS: SAFE
Time 5s | Ax:0.63 Ay:0.88 Az:-11.03 | Temp:23.2/39.0°C | STATUS: SAFE
Time 6s | Ax:0.85 Ay:0.82 Az:-11.73 | Temp:21.1/22.8°C | STATUS: SAFE
Time 7s | Ax:0.21 Ay:0.74 Az:-10.64 | Temp:22.3/30.9°C | STATUS: EVACUATE
⚠️ Path A blocked → Choosing Path B
🚨 EMERGENCY WARNING: ACTIVATING SMART BUILDING ALARMS 🚨
Time 8s | Ax:0.83 Ay:-0.17 Az:-10.70 | Temp:24.2/34.0°C | STATUS: SAFE
Time 9s | Ax:0.34 Ay:0.97 Az:-11.09 | Temp:34.2/37.7°C | STATUS: SAFE

```

In []: